

Image analysis for the microfluidic study of ice-nucleating particles



Usaamha Azeem

University of Leeds

School of Physics and Astronomy

Supervisor: Dr S. A. Peyman

Submitted in accordance with the requirements for a research project
for the degree of

Combined Master of Physics and Bachelor of Science

May, 2023

Declaration of Academic Integrity

I am aware that the University defines plagiarism as presenting someone else's work, in whole or in part, as your own. Work means any intellectual output, and typically includes text, data, images, sound or performance.

I promise that in the attached submission I have not presented anyone else's work, in whole or in part, as my own and I have not colluded with others in the preparation of this work. Where I have taken advantage of the work of others, I have given full acknowledgement. I have not resubmitted my own work or part thereof without specific written permission to do so from the University staff concerned when any of this work has been or is being submitted for marks or credits even if in a different module or for a different qualification or completed prior to entry to the University. I have read and understood the University's published rules on plagiarism and also any more detailed rules specified at School or module level. I know that if I commit plagiarism I can be expelled from the University and that it is my responsibility to be aware of the University's regulations on plagiarism and their importance.

I re-confirm my consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to monitor breaches of regulations, to verify whether my work contains plagiarised material, and for quality assurance purposes.

I confirm that I have declared all mitigating circumstances that may be relevant to the assessment of this piece of work and that I wish to have taken into account. I am aware of the University's policy on mitigation and the School's procedures for the submission of statements and evidence of mitigation. I am aware of the penalties imposed for the late submission of coursework.

Signed : _____ Date _____

Usaamha Azeem

© 2023 The University of Leeds and Usaamha Azeem.

Acknowledgements

I would like to thank my supervisor Dr Sally A. Peyman for her encouraging approach to guiding me through my project and dissertation. I would also like to extend my thanks to Dr Mark D. Tarn for his generosity in assisting me throughout my project and I'd like to acknowledge my academic tutor Dr Gavin Burnell for being a guiding light throughout my degree. Lastly, I would be remiss in not mentioning my loving family for their everlasting support.

Abstract

Ice-nucleating particles (INPs) affect the physical properties of clouds and hence can influence the Earth's climate. It is common to study INPs using microfluidic apparatus, but there remains a computational gap to be bridged regarding the methods by that images of the microfluidic devices are analysed. This report will discuss the steps taken to create a computer program that can analyse results from the Lab-on-a-Chip Nucleation by Immersed Particle Instrument (LOC-NIPI) used by Tarn *et al.* [1]. The key metrics for analysing droplets in the LOC-NIPI included the total droplet count, the number of droplets that froze when passed over a cold plate, and the droplet diameter and velocity. Gauging how many droplets froze was important to find the extent to which different INPs had the ability to alter the freezing points of cloud droplets. The metrics were extracted using the Python programming language and the image analysing package, *OpenCV*. The program devised proved effective in counting droplets, their freezing events, velocities and diameters. The reference values for diameter from Tarn *et al.* [1] were in the range of 70 - 100 μm , after analysis on a test case, a diameter of $90 \pm 6\mu m$ was found. However, the large uncertainty meant that around 7% of the information was lost. It was concluded that a camera with a greater resolution would mitigate this. The reference velocity was approximately $11mm s^{-1}$, which was concordant with the analysis program. The droplet and frozen droplet count were verified manually, and the program performed as expected, with all droplets and freezing events identified.

CONTENTS

1	Introduction	1
1.1	Ice-nucleating particles	1
1.1.1	Effects of INPs on climate	1
1.1.2	Modes of ice-nucleation	1
1.1.3	Cloud seeding	2
1.2	Microfluidic apparatus	4
1.3	Imaging challenges	5
1.4	Aims	6
1.4.1	Objectives	6
2	Literature Review	7
2.1	Droplet morphometry and velocimetry	7
2.2	ImageJ for image analysis	10
2.2.1	Droplet diameter determination	10
2.2.2	Droplet tracking	11
2.3	Neural network methods	12
3	Methodology	14
3.1	Pre-processing of the videos	14
3.1.1	Optimal video conditions	15
3.2	Droplet counting and tracking	16
3.2.1	Droplet detection	16
3.2.2	Channel width calculation	17
3.2.3	Distance/velocity calculation	18
3.2.4	Diameter calculation	19

CONTENTS

3.3	Frozen droplet detection	19
3.4	Graphical user interface	20
4	Results & Discussion	23
4.1	Future directions	25
4.1.1	Calibration methods	25
4.1.2	Video capture standardisation	26
4.1.3	Future program implementations	26
5	Conclusion	28
6	Appendix	29
	References	45

LIST OF FIGURES

1.1	Schematic showing the different routes of ice-nucleation in the different cloud phases. Most nucleation events occur in mixedphase clouds; the three main heterogeneous nucleation mechanisms are deposition, contact and immersion. There is limited evidence for condensation freezing, and it is accepted as a loose variation of immersion and deposition freezing [2, 3]. Image retrieved from Kanji <i>et al.</i> [4].	3
1.2	Imaging of a cold stage-based droplet freezing assay showing a 40 droplet capacity.	4
1.3	2D visualisation of LOC-NIPI chip. Images retrieved from Tarn <i>et al.</i> [1].	5
2.1	DMV image processing from Basu <i>et al.</i> [5].	8
2.2	An example of how the CHT works using loci, where the loci of circles overlap marks the centre of the detected circle. Image retrieved from Ribeiro <i>et al.</i> [6].	10
2.3	A bimodal histogram showing the intensities of pixels, the Otsu threshold will lie between the background and foreground intensity classes. Image retrieved from Szelag <i>et al.</i> [7].	11
2.4	Droplets and partial droplets that were detected by the CHT. Image retrieved from Zantow <i>et al.</i> [8].	12
2.5	Multiple droplets of varying sizes proving a difficult resolve when simultaneously in a frame. Image retrieved from Rutkowski <i>et al.</i> [9].	13
2.6	Irregularly shaped droplets in continuous flow. Image retrieved from Hadikhani <i>et al.</i> [10].	13
3.1	Greyscale images of footage from the LOC-NIPI channel.	14

LIST OF FIGURES

3.2	Two ROIs where A was the region before the cold plate where the velocity and diameter measurements were taken. B was the region where the cold plate lay, i.e., where the droplets froze.	15
3.3	An example of footage where the channel is obscured by shadows (condensation), making for difficult frozen droplet detection.	15
3.4	An example where two droplets were in the column at once.	16
3.5	Pixel visualisation of a droplet where intensities varied within the droplet (non-neighbouring pixels).	17
3.6	Pixel visualisation of a droplet where intensities varied within the droplet (neighbouring pixels).	18
3.7	An example of how the channel width calibration process was carried out, with a total of six points sampled, three on each wall, of which an average width was then calculated.	18
3.8	Six sample pixels on the cold-plate region of the channel were selected, with the text corresponding to the greyscale value of the points.	20
3.9	Eight sample pixels of a frozen droplet selected for its greyscale value.	20
3.10	Freezing event detection.	21
3.11	The completed program with GUI showing extraction of the key metrics.	22
4.1	An example of the exported data in a CSV file, the last row represents the total counts and the average droplet velocities and diameters.	23
4.2	Least squares and RANSAC regression graphs for the dataset of distance against frame count.	25

LIST OF TABLES

1.1	The three main types of clouds and their physical properties [4].	2
2.1	Results of CHT droplet detection from [8].	10

Chapter 1 : Introduction

1.1 Ice-nucleating particles

INPs are aerosols that can originate from many different sources; some include natural biological sources such as desert dust, sea spray and vegetation, and even anthropogenic sources such as chemicals or burning fossil fuels. Once natural sources of INPs find their way into the atmosphere via wind erosion or the combustion process, they can dictate cloud formation, precipitation and electrification; hence, it is important to study INPs. [2, 4].

INPs catalyse the freezing of microscopic cloud droplets; meaning, INPs raise the temperature at which droplets freeze, the average droplet in the cloud without an INP particle can exist in a supercooled liquid state as low as approximately -38°C , and as temperatures drop below, homogenous freezing occurs. INPs cause droplets to freeze heterogeneously at a higher temperature, Tarn *et al.* [1] saw ice nucleation at temperatures as high as approximately -1°C .

1.1.1 Effects of INPs on climate

INPs can control the lifetime of clouds, but clouds help regulate the energy balance of the Earth's climate by absorbing or reflecting solar radiation and maintaining the hydrological cycle [2]. The hydrological cycle describes the way in which water moves dynamically between the Earth's surface and the atmosphere; this cycle helps maintain the planet's water supply and helps regulate the Earth's climate. The cycle begins with water evaporating from rivers, oceans, and lakes due to sunlight; then, it enters the atmosphere. Soon after, the water enters the troposphere, depending on the conditions (such as which phase of cloud it is in), it exists either as a supercooled liquid or a vapour. This is where INPs can cause the supercooled water droplets to freeze and cause early precipitation.

1.1.2 Modes of ice-nucleation

The modes of ice nucleation vary depending on the phase of the cloud. There are three main phases of clouds, their compositions vary due to their altitudes, Table. 1.1. The formation of ice crystals in clouds falls into two main nucleation categories: homo-

1.1 Ice-nucleating particles

Altitude/ m	Cloud Type	Composition	Temperature/ $^{\circ}C$
≤ 2000	Liquid phase	Droplets	≥ 0
2000 - 7000	Mixed phase	Supercooled droplets and ice crystals	0 to -38
5000 - 14000	Ice phase	Ice crystals	≥ -38

Table 1.1: The three main types of clouds and their physical properties [4].

genous and heterogeneous nucleation, Figure. 1.1 shows the different phases of clouds and the available mechanisms of freezing within them. Homogenous nucleation occurs in clouds of the highest altitude at a temperature below $-38^{\circ}C$; this freezing occurs spontaneously and in the absence of any INPs. Heterogeneous ice nucleation occurs when catalysed by an INP; other factors that determine the success of ice nucleation include the temperature or relative humidity (RH). A lower temperature, as well as a higher RH, means a greater probability of ice nucleation occurring. This means the specific mechanism in which the INPs instigate freezing depends upon the conditions, i.e., the temperature, the RH and the altitude.

There are three main modes of heterogeneous nucleation. Deposition ice nucleation: liquid water is absent in this case; supersaturated vapour deposits on the surface of an INP as ice nucleation occurs, this occurs at an RH of more than 100%. This nucleation mode is most prominent for forming ice crystals in clouds of higher altitudes, i.e., ice phase clouds [2, 11]. Contact freezing occurs when the INP collides with the droplet of water, which can occur at much higher temperatures than deposition freezing [4]. The final mode of ice nucleating is immersion nucleation; this is when the INP is immersed in a supercooled droplet, this mode of nucleation is considered to be the most common route of ice formation in mixed-phase clouds. Immersion nucleation is the mode of freezing that was investigated in Tarn *et al.* using the LOC-NIPI apparatus.

1.1.3 Cloud seeding

Contrary to the climate risks discussed, there is an incentive to understand INPs better to leverage the precipitation-changing abilities of the INPs to allow for cloud seeding. Countries with arid regions, such as Ethiopia, have explored cloud-seeding approaches

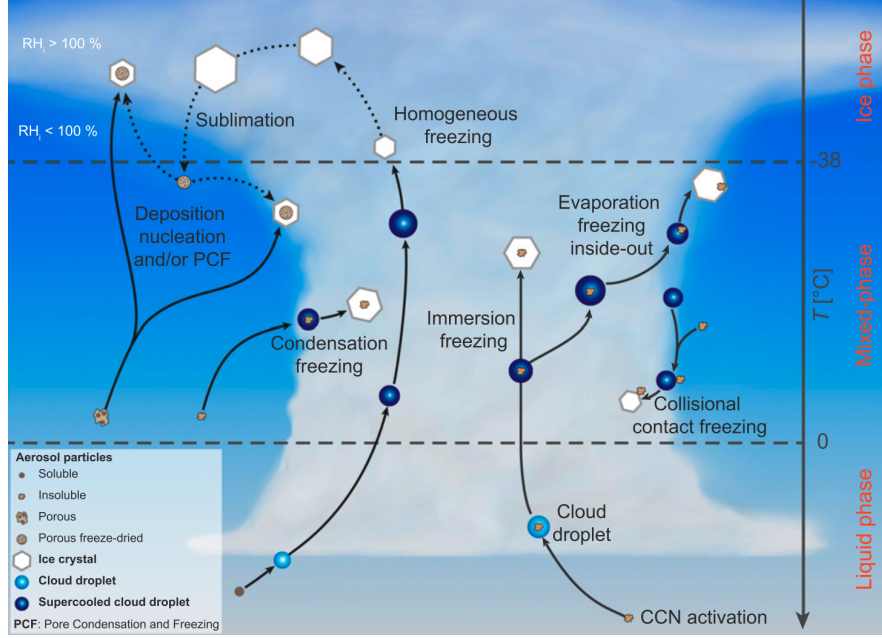


Figure 1.1: Schematic showing the different routes of ice-nucleation in the different cloud phases. Most nucleation events occur in mixedphase clouds; the three main heterogeneous nucleation mechanisms are deposition, contact and immersion. There is limited evidence for condensation freezing, and it is accepted as a loose variation of immersion and deposition freezing [2, 3]. Image retrieved from Kanji *et al.* [4].

to force precipitation in clouds to tackle the scarcity of water [12]. Under the right conditions (e.g., wind speed under 1.5 ms^{-1} and a cloud base height less than 1700 mm) in spring and autumn seasons, cloud-seeding has proven to increase ground-based precipitation measurements [12]. Although, in recent times, research articles supporting the effectiveness of cloud seeding are surfacing, it is a highly debated topic on whether it is feasible [13].

However, there are some setbacks to cloud seeding; there needs to be more research on how much cloud seeding can affect the climate of neighbouring regions and its effects on local ecology. Tarn *et al.* [1] used *Pseudomonas syringae* bacteria (Snomax®) in their experiments, a dead bacteria commonly used in ski resorts to create artificial snow to ensure good coverage of snow on ski slopes. It was concluded by Lagriffoul *et al.* [14] that the *Pseudomonas syringae* bacteria (Snomax®) has low-level health risks and risk of potential allergy in some individuals.

By better understanding the processes and causes by which the INPs alter the properties of clouds, we can have better climate predictability. However, we have still not been able to accurately model cloud formation and precipitation, leading to significant gaps between simulated and observed effects of INPs on cloud droplets [2].

1.2 Microfluidic apparatus

Tarn's group used the LOC-NIPI apparatus to investigate immersion nucleation. The LOC-NIPI method allowed a high throughput of droplets which allowed thousands of droplets to be investigated, Figure. 1.3a. Tarn's group also experimented with a cold-stage-based droplet freezing assay (CSFA). This apparatus offered a low throughput for inspecting droplets; only tens or hundreds could be analysed simultaneously after being manually located on the plate, Figure. 1.2. The CSFA also had droplets on the scale of microlitres allowing more of a chance for the droplets to contain impurities. However, the LOC-NIPI had droplets of a few picolitres and hence reduced the chance of impurities. For these reasons, it was decided that the LOC-NIPI was an ideal setup for investigating INPs.

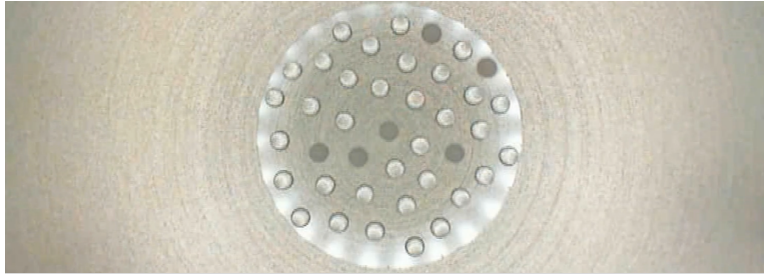
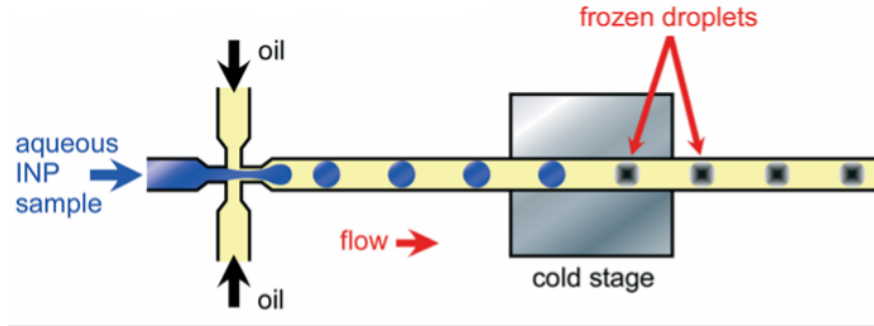


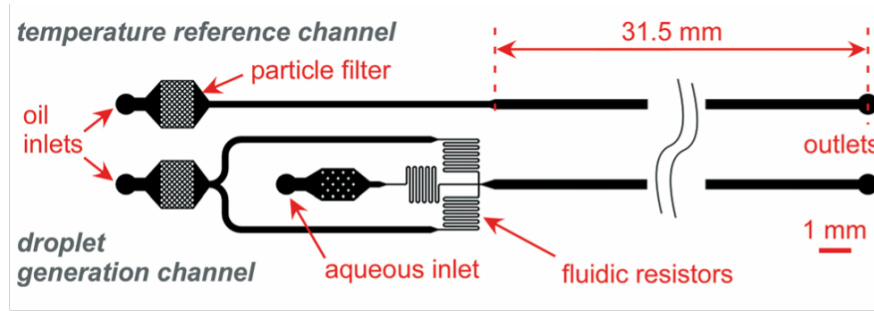
Figure 1.2: Imaging of a cold stage-based droplet freezing assay showing a 40 droplet capacity.

The experimental apparatus comprised a channel placed on a chromium-coated glass slide. The aqueous INP sample was injected into the channel after being segmented into aqueous droplets in a carrier oil, Figure. 1.3a. The microfluidic chip had a section of the channel placed on top of a cold plate. Cold plate temperatures were varied to test the freezing point of the INP samples. The oil temperature was then indirectly measured to match the frozen droplets to their freezing point, Figure. 1.3b.

Tarn *et al.* [1] tested the LOC-NIPI apparatus with pure water, birch pollen extract



(a) Mixture of oil and the INP solution entering the main channel after being segmented into droplets.



(b) 2D visualisation of LOC-NIPI parallel channels, both channels were placed on the cold plate so the upper channel could be used as a non-intrusive proxy for measuring the oil temperature.

Figure 1.3: 2D visualisation of LOC-NIPI chip. Images retrieved from Tarn *et al.* [1].

and *Pseudomonas syringae* bacteria (Snomax®). A Phantom Miro Lab 120 high-speed camera (Vision Research Ltd., Bedford, UK) was used to capture the movement of the droplet through the microfluidic channel.

1.3 Imaging challenges

With thousands of droplets passing through the microfluidic channel, the number of droplets that froze needed to be gauged, along with their velocities and diameters. The volume of the droplet and the velocity at which the droplet passes through the microfluidic channel influenced the cooling rate of the droplet, which then affected the nucleation time. Hence, tracking these metrics further to investigate the effects of droplet size/velocity and maintain experimental reproducibility was essential.

Tarn *et al.* [1] used *ImageJ* to manually measure droplet diameters and velocities. The velocities were measured by carefully selecting the same section of the droplet in each frame of the video; with hundreds of droplets and thousands of frames, this time-intensive method had room for automation. Manually tracking droplets meant there was still information to be extracted, i.e., not every droplet in every frame was analysed, and often a measurement was taken at intervals. With an automated computational approach, the droplets could be constantly tracked and measured accurately, experiments could be held for longer, and higher throughput of droplets meant a quicker analysis rate. Hence, this paper will describe the process that was carried out to make an image analysis of the LOC-NIPI microfluidic device automated, as well as investigate methods that other groups carried out to analyse microfluidic images.

1.4 Aims

To devise a robust and automated computer program that could automatically analyse droplets and extract the three primary metrics (fraction of frozen droplets, diameter, and velocity) of INP droplets in continuous-flow microfluidic channels.

1.4.1 Objectives

- Accurately determine the number of droplets that froze.
- Classify droplets by measuring their diameters.
- Measure the velocity of the droplets.
- Ensure the main three criteria were accurately measured inclusive of all exceptional cases.

Chapter 2: Literature Review

2.1 Droplet morphometry and velocimetry

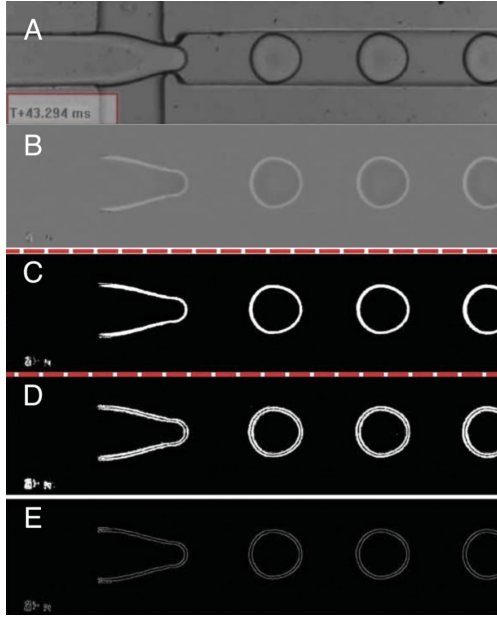
Basu *et al.* [5] introduced the concept of Droplet Morphometry and Velocimetry (DMV); it is an algorithm that automates the tracking of the shape and velocity of a droplet over multiple frames. Droplets appeared translucent on video when passing through a microfluidic channel; they leveraged the dark boundary between the droplet and the carrier fluid (which arose due to the reflective index difference between the droplet and fluid) to track the droplet, seen in Figure. 2.1a image A.

Basu *et al.* [5] used a picture of the empty channel to calibrate the background subtraction. Colour inversion was applied, which resulted in a prominent bright border around the droplet; as the droplet was translucent, the inner body of the droplet blended into the background. The group investigated three edge detection methods, intensity thresholding or gradient-based algorithms such as the Sobel and Canny methods, shown in Figure. 2.1a in images C, D and E.

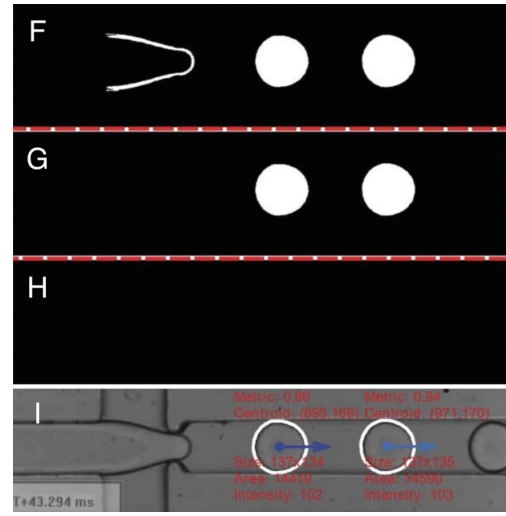
The Sobel edge detection method is a computationally simple and fast method of carrying out edge detection; it uses two convolution filters to compute the gradient direction and magnitude for each pixel. Once gradients have been identified, thresholding can be done to identify edges of interest. The Sobel edge detector can be sensitive to noise and have issues with identifying faint edges.

The Canny method is more resistant to noise and can identify faint/thin edges in the image, it works in the same way the Sobel edge detector does, but it has a few more steps. First, the Canny edge detector applies the Sobel edge detector, and then edges are thinned to keep only the local maxima in the gradient magnitude that point in the direction of the edge. Double thresholding is then used to categorise pixels according to their gradient strengths, and then edge hysteresis is used to link firm edges to feinter edges [15]. Edge hysteresis is a method of thresholding gradients, two automatically calculated thresholds are used. Values below the lower threshold are deleted, values between the thresholds are classified as weak edges, and values above the upper threshold are classified as strong edges; weak edges are only saved if they connect to a strong edge. The Canny edge detector is more complex and computationally expensive than intensity thresholding and the Sobel edge detector; however, it can be used to identify tightly packed droplets or faint edges [5].

2.1 Droplet morphometry and velocimetry



(a) A is a greyscale image of the microfluidic channel containing droplets. B is an image after background subtraction (an image of an empty channel was used to calibrate the background subtractor). C, D and E show the different droplet edge identifiers, C being intensity thresholding, D being the Sobel edge detector, and E being the Canny edge detector. In E, the Canny detector can detect finer lines, i.e., the inner and outer wall of the droplet border being detected separately.



(b) Image F shows the circles filled between in their boundaries, Images F, G and H are also images that have been thresholded by circle metric values of 0.1, 0.5 and 0.99, respectively. Image I shows the circle detection of both droplets with circle metric values of 0.86 and 0.84, respectively.

Figure 2.1: DMV image processing from Basu *et al.* [5].

2.1 Droplet morphometry and velocimetry

The contrast between the background and droplet intensities was high, so simple intensity thresholding was used instead of the Sobel and Canny edge detecting methods. It is still important to note the computational capability of the Canny and Sobel methods, for images where droplets are harder to resolve, edge detection methods are resources worth having. After thresholding, the shapes with any space enclosed by boundaries were filled, and then shape discrimination was done with a circle metric; the circle metric was as Equation. 2.1. The area enclosed by the boundary and the perimeter of the boundary would then be used in this equation to give a value. A perfect circle would yield a value of 1, Equation. 2.2. It was mentioned that the shape discrimination process could utilise the circle Hough Transform (CHT), but it was deemed too computationally expensive.

$$\text{Circle metric} = \frac{P^2}{4\pi A} \quad (2.1)$$

$$\text{Circle metric} = \frac{(2\pi r)^2}{4\pi^2 r^2} = 1 \quad (2.2)$$

The CHT works by using a Canny edge detector to find all edges in an image; an accumulator is then initialised to keep track of all circle centres, (x, y) and radii, r . The CHT will then draw loci of circles for each edge that exists from the Canny edge detector. While drawing the loci of circles, each locus's minimum and maximum diameter is varied until the common point of intersection between all circles in the locus is maximised, Figure. 2.2. The shared point of intersection is the centre (x, y) of the detected circle, and the radius is the distance between the centre of a locus and the point at which the intersection is maximised, i.e., the centre of the detected circle.

The CHT would mitigate the use of any morphological operations. i.e., shape fill and the use of circle metric. However, using only the CHT would mean information would be lost regarding the circle perfectness of the droplet; for the cases of tracking and measuring the diameter of the droplet, the CHT by itself would suffice, albeit at lower accuracy.

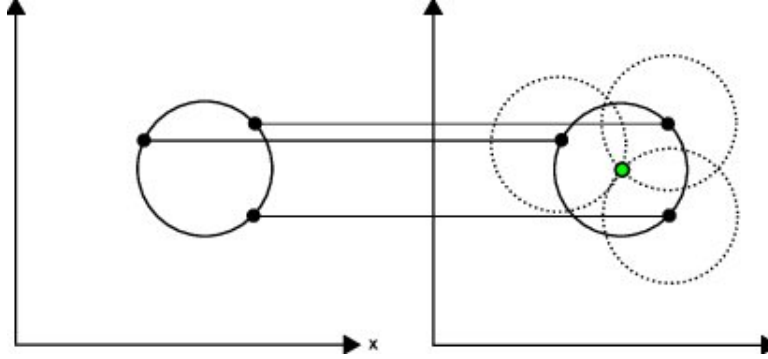


Figure 2.2: An example of how the CHT works using loci, where the loci of circles overlap marks the centre of the detected circle. Image retrieved from Ribeiro *et al.* [6].

2.2 ImageJ for image analysis

2.2.1 Droplet diameter determination

Zantow *et al.* [8] analysed images from a microfluidic channel containing encapsulated enzymes to determine droplet diameters using CHT methods along with the ImageJ 'particle tracker' plugin to investigate droplet behaviour as they move through channels. Zantow *et al.* experimented with using different magnifications, at a 20x magnification, they achieved a resolution of 0.2 microns per pixel, compared to 4-5 microns per pixel from Tarn *et al.* [1]. Images were pre-processed using a Sobel edge detector, as discussed in Basu *et al.* [5]. After Sobel edge detection, the images were converted to binary using the Otsu method [16], which was based on automatic thresholding of the intensity histogram. As the intensity histogram was bimodal, the threshold lay between the two modes, Figure. 2.3. The CHT was then applied to the binary image using ImageJ plugins, Figure. 2.4.

Magnification	Manual count	Hough Transform	% Difference
4x	3219	4444	38.1
10x	326	403	23.6
20x	46	48	4.3

Table 2.1: Results of CHT droplet detection from [8].

As seen in Table. 2.1, the CHT performed best at greater magnifications, the

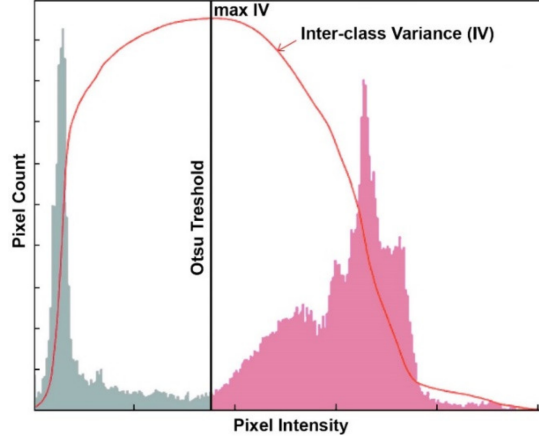


Figure 2.3: A bimodal histogram showing the intensities of pixels, the Otsu threshold will lie between the background and foreground intensity classes. Image retrieved from Szelag *et al.* [7].

difference between the CHT count and the manual count differed the least at greater magnifications due to the greater pixel resolution. The percentage difference shows that a lower magnification caused the CHT to produce more false circles. The manual count only included full droplets, but the CHT counted partial droplets also; hence the CHT often overestimated the droplet count at all magnifications.

2.2.2 Droplet tracking

First, Zantow blurred all frames with a Gaussian blur to smooth all parts of the image, and then a spatial convolutional sharpening filter was used to sharpen the edges of the droplets. The same Otsu method from the droplet diameter measurement was used to threshold and made a binary image. The 'analyse particle' feature in ImageJ was used to find and fill objects based on their edges, then from the object filling, the area was found and a circle metric similar to Equation. 2.1 was used to gauge the circularity of detected droplets.

Similar to Tarn *et al.*, Zantow *et al.* [8] used the ImageJ 'particle tracker' plugin, which consisted of manually tagging the droplet in each frame. The calculations (distance and velocity) were automatic once the pixel-to-distance conversion had been initialised. The method suffices for tens of frames, but it is pedantic and inefficient for thousands. This method is enough to approximate the velocity of droplets, but

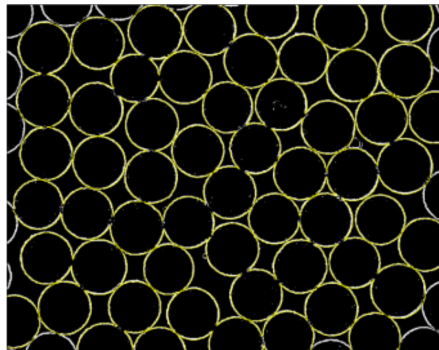


Figure 2.4: Droplets and partial droplets that were detected by the CHT. Image retrieved from Zantow *et al.* [8].

calculating an average speed based on thousands of particles would have been difficult.

Some extensions can be used on ImageJ to auto-track particles, particularly *Track-Mate* [17]; however, such macros work best with only short-form videos and singular particles. Most tracking tools in ImageJ are primarily used to track particles that do not leave the frame; however, when videos start to contain thousands of entering and leaving the frame, it becomes increasingly difficult to track, and hence a solution for analysis out of ImageJ is most required in the case for LOC-NIPI analysis.

2.3 Neural network methods

There is an abundance of machine-learning-based approaches to microfluidic image analysis, especially using deep neural network (DNN) models. In Rutowski *et al.* [9], there were often many droplets in a frame with some droplets being out of focus and flow rates in excess of 100 droplets s^{-1} , Figure. 2.5.

In the LOC-NIPI experiment, flow rates were typically 1 or 2 droplets s^{-1} . This provided ease in detection due to less processing required per frame and mitigated the possibility of out-of-focus droplets. Although the need for complex machine-learning-based image detection methods is unnecessarily complex for our case, there is room for implementation, considering that many of the properties measured are the same in this analysis. This approach may be useful in (CSFA), where there are several droplets in a single frame, and the level of image clarity may not be consistent on all of the droplets.

In Hadikhani *et al.* [10], microfluidic droplets were of irregular shape, Figure. 2.6,

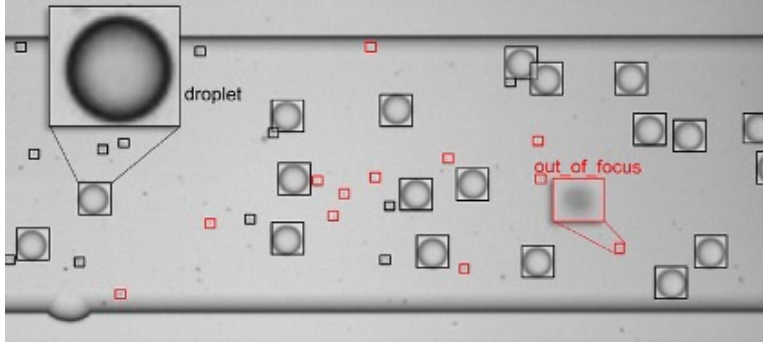


Figure 2.5: Multiple droplets of varying sizes proving a difficult resolve when simultaneously in a frame. Image retrieved from Rutkowski *et al.* [9].

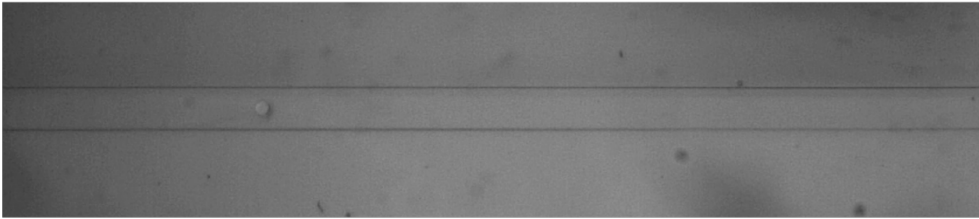
hence, simple circle detection did not suffice, and a machine learning approach of *scikit-image* for edge detection was used. This would be a potential avenue for implementation if the CHT struggled to resolve droplets accurately.



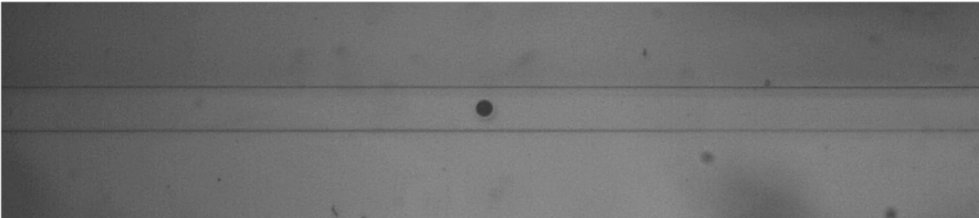
Figure 2.6: Irregularly shaped droplets in continuous flow. Image retrieved from Hadikhani *et al.* [10].

Chapter 3: Methodology

Image analysis of the videos was carried out using Python and the OpenCV library. The videos that were collected by Tarn *et al.* [1] were in a greyscale format, and a Phantom Miro Lab 120 high-speed camera (Vision Research Ltd., Bedford, UK) was used to capture the movement of the droplet through the microfluidic channel Figure. 3.1. The videos were captured at 177 frames per second (fps) which allowed enough frames for the video to contain any freezing events along the droplets' trajectories. The camera needed to have the capacity for a high frame rate as the point of freezing (dendritic growth) was brief, so it was also important that the video had not been decimated (decimation is a form of video compression that deletes frames to reduce file size).



(a) Liquid droplet passing through the channel before being frozen.



(b) Droplet at the point of freezing appearing darker.

Figure 3.1: Greyscale images of footage from the LOC-NIPI channel.

3.1 Pre-processing of the videos

Videos from the high-speed camera captured the full chromium glass slide, which was surplus to the channel, and hence the video was cropped. It was essential to crop the video as chromium slip outside the channel appeared to have dust and other artefacts; these artefacts could cause false detection of droplets.

It was important to encompass the cold plate region when cropping, as droplets only froze when passing over the cold plate, as seen in region B in Figure. 3.2. Inaccurate cropping of the video would result in missing freezing events. Enough space for the droplet to move through the channel before the cold plate was important so that accurate velocity and droplet diameter measurements could be taken, as seen in region A in Figure. 3.2. This was the case because once the droplet had frozen, the crystal formation obscured the droplet, i.e., a droplet that had visually changed would interfere with the detection parameters.

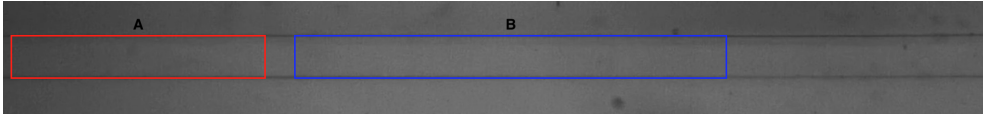


Figure 3.2: Two ROIs where A was the region before the cold plate where the velocity and diameter measurements were taken. B was the region where the cold plate lay, i.e., where the droplets froze.

3.1.1 Optimal video conditions

Optimal experimental conditions for video analysis would mean that the channel background (chromium glass slip) was free of any dust or residues, as it could cause false positives in freeze event detection. It would also include enough contrast between the droplet and the background so that the circumference of the droplet is visible, allowing for better detection of droplets and better droplet diameter measurements. To achieve this, the chip must be clear of any condensation, as this would cause shadows to obscure the droplet, Figure. 3.3.

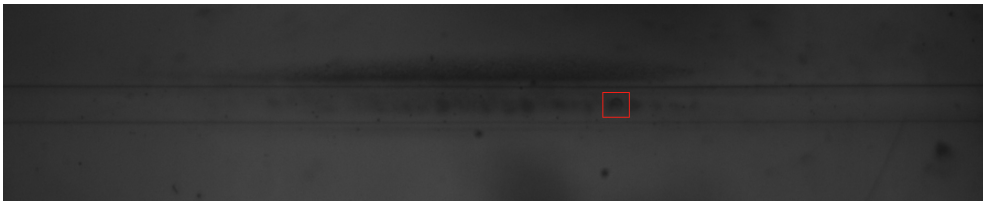


Figure 3.3: An example of footage where the channel is obscured by shadows (condensation), making for difficult frozen droplet detection.

3.2 Droplet counting and tracking

3.2.1 Droplet detection

All frames of the video were converted from BGR to greyscale. The `cv2.createBackgroundSubtractorMOG2()` background subtractor module was used to remove the video background, this isolated any moving objects, and hence the droplets entering the channel could be tracked and counted. The Mixture of Gaussians (MOG2) algorithm models each pixel as a Gaussian distribution, and it is commonly used to separate moving objects from a static background. MOG2 was chosen over its contemporaries due to its ability to deal with shadows and adaptive learning. i.e., any changes in lighting conditions would be negligible.

Once the foreground had been extracted using the MOG2 background subtractor, morphological operations were carried out to clean the foreground of noise. The function `kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))` was used to form an elliptical kernel of size 3x3 pixels, this kernel forms a neighbourhood around each pixel. Then the kernel was used in `foreground_mask = cv2.morphologyEx(foreground_mask, cv2.MORPH_CLOSE, kernel)` to perform the closing morphological operation, which helped remove holes and closed gaps between objects which resulted in image smoothing between pixel neighbours.

After image processing and foreground extraction, a region of interest was selected, region A in Figure. 3.2. A portion of the left-most part of the channel was selected because, occasionally, two droplets were simultaneously in the channel. To reduce the chance of this, only the portion of the channel where the droplets entered was used as there were normally a few seconds before the next droplet entered Figure. 3.4.

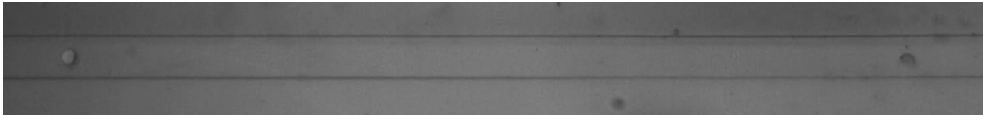


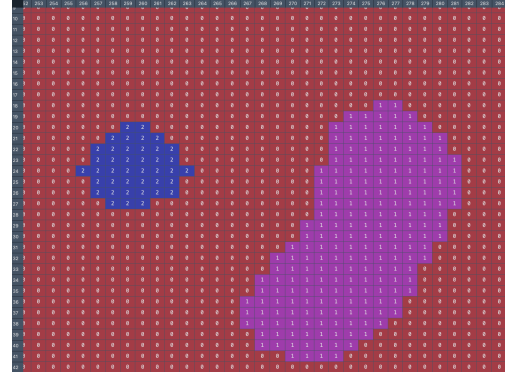
Figure 3.4: An example where two droplets were in the column at once.

The function `cv2.connectedComponentsWithStats(foreground_Mask, connectivity=8)` was used to analyse the droplets in the foreground; it is commonly used in object tracking and classifies objects based on their neighbours. The binary foreground image was analysed to classify droplets based on their pixels; if each pixel's neighbours were

deemed moving with the pixel, it was counted as one object. If there was a separation of at least a pixel (either horizontally, vertically, or diagonally) between moving pixels, they were deemed two separate objects and were labelled as such, Figure. 3.5. Figure. 3.6 shows the labelling of the droplet in the foreground; as all pixels were neighbouring, they were seen to be moving together as one object. As seen in Figures. 3.6 and 3.5, two labels can arise from a single droplet, this happens when droplets appear to have a range of greyscale intensities; causing the algorithm to register two labels for each of the intensity classes. Once the droplet had left the ROI, the label count reverted to 0 and the droplet count was incremented assuming that the droplet had left the frame.



(a) Binary image of a droplet with two different intensity classes that weren't neighbouring.



(b) Visualisation of all the pixels in the droplet, as the two intensity classes weren't neighbouring, the pixels in the droplet weren't all labelled the same but instead, 1 and 2.

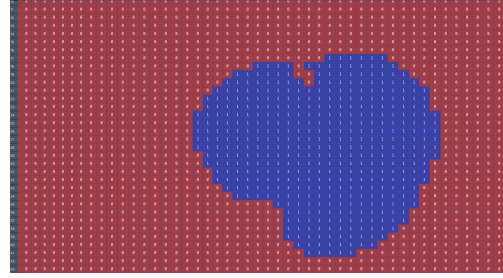
Figure 3.5: Pixel visualisation of a droplet where intensities varied within the droplet (non-neighbouring pixels).

3.2.2 Channel width calculation

The conversion factor of pixel to micron was needed for the velocity and diameter measurements to be converted to microns. As the channel width was known to be 300 microns [1], the pixel width of the channel was used as a conversion metric to microns. Using a click event tracker, a calibration procedure was introduced before any image analysis; this included loading in a frame and allowing the user to identify and click where the channel walls lay, Figure. 3.7. The calibration program prompted three clicks per wall, an average channel wall separation (width) was calculated, and then



(a) Binary image of a droplet with two different neighbouring intensity classes.



(b) Visualisation of all the pixels in the droplet, as the two intensity classes are neighbouring, all pixels in the droplet are labelled the same, 1.

Figure 3.6: Pixel visualisation of a droplet where intensities varied within the droplet (neighbouring pixels).

the micron per pixel value was determined.

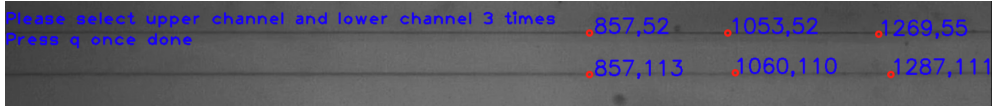


Figure 3.7: An example of how the channel width calibration process was carried out, with a total of six points sampled, three on each wall, of which an average width was then calculated.

3.2.3 Distance/velocity calculation

When the droplet was detected, the cluster's centre was determined and hence the x and y coordinates. With this, as the pixel cluster moved through each frame, the distance travelled was monitored by triangulating the distance between the centre point (x, y) in each frame. Now that the distance travelled by the droplet per frame was known, the velocity of the droplet could be calculated using the capture frame rate of the camera. The capture frame rate was 176 fps, so the time interval between each frame corresponded to $1/176$ th of a second. Using the simple distance and velocity relation, the velocity was calculated for the droplet as it moved through each frame. When the droplet had left the region of interest, the label count (droplet count) from the `cv2.connectedComponentsWithStats()` function would show that the droplet was

not present, and hence the distance parameter would be reset to 0.

3.2.4 Diameter calculation

Image processing and cleaning were carried out in the same fashion that the droplet counting/velocity tracking was carried out in, i.e., the same function was used. The `cv2.HoughCircles()` CHT function was used to detect circles in the foreground binary image.

With the output from the CHT function, the circle with the maximum radius was found using `max_circle = max(self.circles[0], key=lambda x:x[2])` where `self.circles` was a nested list of detected circles containing their centres (x, y) and radius, r .

3.3 Frozen droplet detection

Before detecting freezing events, it was important to decipher the area of the channel that covered the cold plate. For this, videos had to be initially checked by the user to determine where the freezing events occurred. Once the region of interest had been deciphered, the frame could be accordingly cropped for analysis to be carried out, region B in Figure. 3.2. Noise and granularity were blurred out of the frame using `cv2.medianBlur()`, which applied median filtering to the pixels by replacing each pixel with the median of its neighbouring pixels. Next, the colour threshold needed to be discovered; before any colour sampling, the frame was converted to greyscale using `cv2.COLOR_BGR2GRAY`. This assigned all pixels a colour value between 0 and 255, with 0 being the darkest shade of black and 255 the brightest white. Then, using the greyscale frames, the channel background was sampled using an eye dropper function, Figure. 3.8. To ensure an accurate threshold, a freezing event had to be sampled; this meant the user had to input a screen capture of a freezing event that had been identified manually. The freezing event would then be sampled using the eye dropper function used previously, Figure. 3.9. The functions allowed the background and freezing events to be sampled multiple times to gain an average. Then, the midpoint greyscale value of the background and freezing event was calculated; this was the threshold used, Equation. 3.1.



Figure 3.8: Six sample pixels on the cold-plate region of the channel were selected, with the text corresponding to the greyscale value of the points.

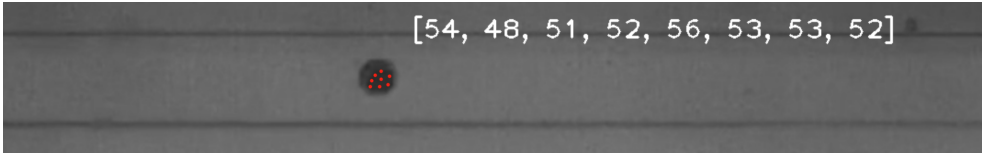


Figure 3.9: Eight sample pixels of a frozen droplet selected for its greyscale value.

$$Threshold = \frac{background\ intensity + frozen\ droplet\ intensity}{2} \quad (3.1)$$

Each frame in the video was then looped through, and processed as above, i.e., blurring and region of interest cropping. `cv2.findContours()` was used to contour around any object with a greyscale colour below the threshold, Figure. 3.10. The contours were verified by their areas, as it was known that the diameters were approximately $100\ \mu m$ and an approximate area of $10000\ \mu m^2$ [1]. From the calculation of the channel width, the area of the frozen droplet could be estimated as between $5000\ \mu m^2$ and $15000\ \mu m^2$. This allowed enough tolerance as ice formation sometimes only appeared on part of the droplet, so the area covered could be less than that of a circle with a $100\ \mu m$ diameter. The upper limit of $15000\ \mu m^2$ (equivalent to the area of a circle with a diameter of $150\ \mu m$) attempted to mitigate the chances of detecting shadows and other artefacts.

Another failsafe to avoid double counting frozen droplets was used; the frozen droplet detector would be compared to the main droplet count, i.e., if the frozen droplet detector had claimed to found a frozen droplet, the history of the main droplet count would be checked to ensure another droplet had entered the channel; this also mitigated chances of detecting artefacts, or double counting a freezing event.

3.4 Graphical user interface

The methods were combined to form an all-in-one graphical user interface (GUI) that could measure droplet velocities, diameters, counts and freeze counts. The GUI dis-



(a) Binary threshold applied to the frames, masking the lighter background. Only the point at which dendritic growth appears was not masked due to it being dark and hence, below the threshold.



(b) Detection of the droplet at the point of freezing.

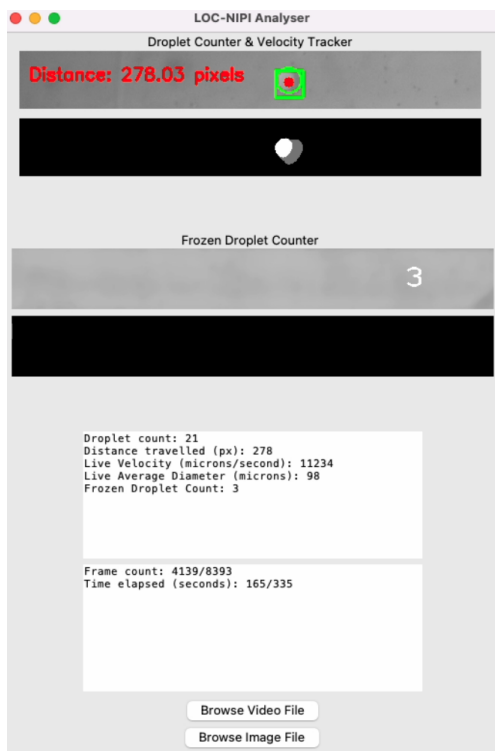
Figure 3.10: Freezing event detection.

played live metrics in two text boxes; the first text box contained the droplet/frozen droplet count, distance travelled by the droplet, live velocity, and average diameter. The second text box contained the frame count and time elapsed. The live velocity was of the droplet actively passing through region A in Figure. 3.2, the diameter was also calculated from droplets in this region; the values were represented as the rolling average of all the droplet diameters measured.

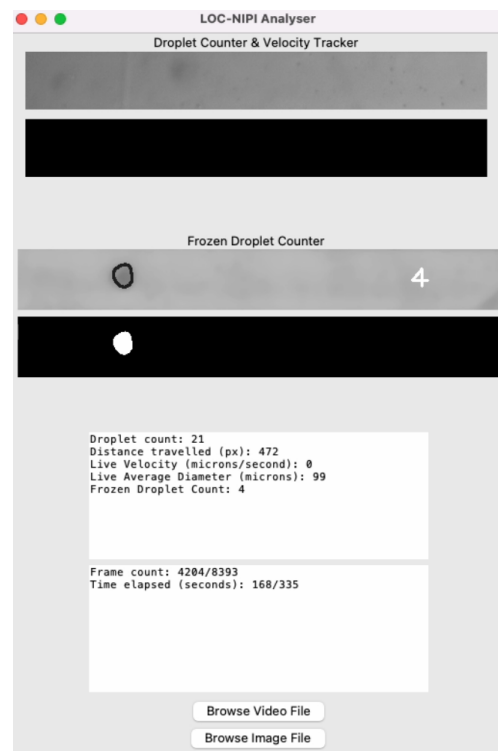
The upper two frames of the GUI seen in Figure. 3.11, were the original and binary frames of the leftmost part of the channel, i.e., region A in Figure. 3.2. The lower two frames were the original and binary frames of the cold plate region, i.e., region B in Figure. 3.2.

The two buttons in the GUI allowed the selection of a video and an image; the video file captured all the droplets moving through the channel. The image file was a pre-selected frame for the intensity sampling of the freezing event, as discussed in section 3.3. Once playback of the video had concluded, all data was exported to a CSV file, including the final droplet and freeze count, the average diameter and the average droplet velocity.

3.4 Graphical user interface



(a) GUI showing the tracking of a droplet, whilst taking its count and measuring its velocity.



(b) GUI showing the detection of a freezing event.

Figure 3.11: The completed program with GUI showing extraction of the key metrics.

Chapter 4: Results & Discussion

As discussed in section 3.4 all results were exported as a CSV file, Figure. 4.1. With the analysis of a sample video using the program, 48 droplets were detected and 21 of them had frozen. The counts agreed with the true values, however, a discrepancy in the detection of freezing events could occur if the cold plate ROI was selected incorrectly, i.e., if the ROI was cropped too short, the detection of some freezing events could be missed. As cold plate region varied depending on the camera and cold plate placement, it often meant that time had to be spent watching the video to make a rough estimate of where the cold plate lay. The process could be refined by adding markers on the chromium slip indicating where the ends of the cold plate lie, hence mitigating the inaccuracies in detecting freezing events.

Frame count:	Time elapsed (seconds):	Total droplet count:	Total frozen droplet count:	Live Velocity (microns/second)	Live Diameter (microns):
1/8393	0.04/335.72	0	0	0	0
2/8393	0.08/335.72	0	0	0	0
3/8393	0.12/335.72	0	0	0	0
4/8393	0.16/335.72	0	0	0	0
"	"	"	"	"	"
"	"	"	"	"	"
"	"	"	"	"	"
"	"	"	"	"	"
8150/8393	326.0/335.72	47	21	10507	126
8151/8393	326.04/335.72	47	21	10521	85
8152/8393	326.08/335.72	47	21	10533	80
8153/8393	326.12/335.72	47	21	10522	78
"	"	"	"	"	"
"	"	"	"	"	"
"	"	"	"	"	"
"	"	"	"	"	"
8390/8393	335.6/335.72	48	21	10080	95

Figure 4.1: An example of the exported data in a CSV file, the last row represents the total counts and the average droplet velocities and diameters.

The average diameter of the droplets in the sample video was $95 \mu m$, this was in agreement with the quoted value of $70-100 \mu m$ from Tarn *et al.* [1]. However, due to the low magnification of the camera lens, the videos only had a resolution of $4-5 \mu m$ per pixel, compared to the $0.2 \mu m$ per pixel resolution in Zantow *et al.*. This meant there was considerable uncertainty when calculating the droplet diameter; this is especially important as diameters were quoted to the nearest micron in Tarn *et al.* [1]. Hence, with an intrinsic pixel uncertainty of approximately $5 \mu m$, droplet diameters could have

as large as 7% from only the intrinsic pixel uncertainty. There are also other sources of uncertainty, such as the channel width to pixel conversion uncertainty.

Looking at the uncertainty propagation Equation. 4.1 and 4.2, x was the channel width in microns, and Δx was the uncertainty in the channel with which is taken at the standard manufacturer uncertainty of 1. The y was the channel width in pixels, and Δy was the channel measurement uncertainty of 2. The propagation of these two uncertainties yielded 1 pixel = $5.0 \pm 0.2 \mu m$. So where a droplet had a measured diameter of 18 pixels, the true diameter and uncertainty would be $90 \pm 6 \mu m$ as in Equation. 4.3.

$$\Delta z = z \sqrt{\left(\frac{\Delta x}{x}\right)^2 + \left(\frac{\Delta y}{y}\right)^2} \quad (4.1)$$

$$0.2 \mu m = 5 \sqrt{\left(\frac{1}{300}\right)^2 + \left(\frac{2}{60}\right)^2} \quad (4.2)$$

$$6 \mu m = 90 \sqrt{\left(\frac{0.2}{5}\right)^2 + \left(\frac{1}{18}\right)^2} \quad (4.3)$$

Although using a graticule to ease the pixel to μm conversion process and mitigate some of the uncertainty arising in the conversion, most of the uncertainty came from the low pixel resolution. A higher-resolution camera or a magnifying lens on the current camera would allow for diameter readings with less uncertainty.

An average droplet velocity of $10 mm s^{-1}$ was measured using the sample video, and the quoted value from Tarn *et al.* [1] was approximately $11 mm s^{-1}$. The velocity value can vary depending on the region of interest chosen, and as the $11 mm s^{-1}$ value was only an approximation, the measured velocity is acceptable. The variation in droplet velocities is likely to be due to the averaging method used. A superior averaging method was experimented with but still needs to be implemented; the Random sample consensus (RANSAC) method is a machine learning method of performing linear regression on a dataset that contains outliers [18]. The simple averaging method used in the program and the traditional least squares method are highly sensitive to outliers and noise. The RANSAC method works by iteratively selecting sets of sample data points whilst simultaneously selecting best-fit models based on inliers until the inliers are maximised and the best linear fit is found. Figure. 4.2 shows the effect of outliers on the least squares method compared to the RANSAC method.

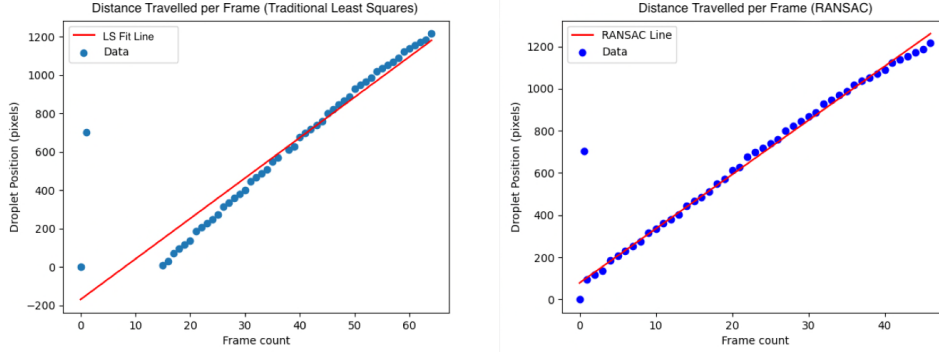


Figure 4.2: Least squares and RANSAC regression graphs for the dataset of distance against frame count.

The velocity calculation only considered part of the trajectory of the droplet, the trajectory in the leftmost part of the channel, i.e., region A in Figure. 3.2; this was in case two droplets were in the frame at once, as seen in Figure. 3.4. The program was limited to counting only one droplet (freezing event), as all labels (contours) not in the foreground were counted as one entity. Ideally, with stricter droplet control, the droplet's trajectory through the full channel could be tracked, providing a larger sample size for velocity calculations and allowing the effects of freezing on the droplet's velocity to be investigated.

4.1 Future directions

4.1.1 Calibration methods

The sampling calibration method of finding a threshold discussed in section 3.3 was sufficient; however, there was room for automation. The Ostu method [16] that Zantow *et al.* [8] used allowed room for automation because there were only two dominant intensities (channel background and freezing event) Figure. 3.1b, the threshold could lie in between the two intensity modes as seen in Figure. 2.3.

Another method of sampling mentioned in Zantow *et al.* [8] consisted of taking the intensity sample of the four corners of the frame; in the case of this analysis, this would be feasible after the cold-plate ROI cropping had been carried out as the corners of the frame would usually have a slightly different intensity compared to the cold plate ROI intensity. Another method of sampling the cold plate ROI would include automatically

sampling all pixels in the ROI and then calculating an average. This said, although the manual calibrating took more time, it allowed more control for the user. i.e., if there were a detection error, the user would at least know whether there were any issues in wrongful calibration.

4.1.2 Video capture standardisation

The quality of the videos varied each time a batch of droplets was recorded; this included some of the videos having significant shadows/artefacts, and sometimes videos would have too dark a background intensity to resolve any droplets. If the videos were standardised to be free of artefacts and had a background intensity above a certain threshold, i.e., 100/255 on the grayscale, it would allow enough gap between the frozen droplet intensity and the background intensity. Enough of a difference between the droplet and background intensity would mean the program could detect droplets more accurately.

Several videos were received for analysis that had been decimated, i.e., files were compressed by discarding frames. Although decimating video files allowed for quicker playback, and more particles could pass through the channel in a specific time frame, it was detrimental to the analysis process. If there were too few frames showing a droplet's trajectory, not only would it mean the risk of missing a freezing event, but it would also mean that the velocity calculations would have been less accurate. Hence, it was important that the camera's 177fps was unchanged in video exporting.

4.1.3 Future program implementations

The program's default capture frame rate was set to 177 fps; however, for the program to be more versatile and compute at different frame rates a dialogue box could be implemented to ask the user for their capture frame rate. For the batch of videos received to examine, all were believed to be captured at 177 fps. An fps that is different from 177 would result in incorrect velocity calculations, e.g., if a video were decimated by deleting every other frame, the velocity computed by the program would be an overestimate by a factor of two, meaning instead of an 11mm s^{-1} velocity, the program would output a velocity of 22mm s^{-1} .

The width of the channel in the program was also set to $300\text{ }\mu\text{m}$, which meant less reusability for microfluidic devices that have a different channel width. A dialogue box

4.1 Future directions

for the user to input their channel width would allow the program to be used to analyse video footage from microfluidic channels with different widths.

Chapter 5 : Conclusion

A program was devised that can automatically, count droplets, freezing events, and measure their diameters and velocities. In the test cases used, the droplet and freezing event counts were true to their actual value. An average diameter of $90 \pm 6\mu m$ was found, and the reference values for diameter from Tarn *et al.* [1] were in the range of 70 - $100\mu m$. The reference velocity was approximately 11mms^{-1} which was similar to the findings (2 s.f.) from the program.

Ideally, the final iteration of the code would be an all-encompassing integrated software so that both the analysis and the operation of the experimental hardware (syringe pumps and cooling stage) can be controlled as one system. This would mean the live video feed being processed in real-time, allowing for a more efficient analysis procedure. The program explained in this paper assumes droplets to be clearly visible with high contrast between the background and droplets; for more obscured droplets in non-standardised video conditions, the program would fare better with a trainable machine learning algorithm, such as the ones from either Hadikhani *et al.* [10] or Rutowski *et al.* [9].

The program is not limited to analysing only INPs but can be used as a foundation for the making of a general tool for the all-in-one microfluidics analysis software similar to ImageJ but instead, directly focused on the image analysis of microfluidics.

Chapter 6 : Appendix

"""

Completed as of 01/05/23

Code was developed on macOS 12.5.1 M1 with Python 3.7.13 and OpenCV 4.1.0.

This script is for analysing droplets in the LOC-NIPI apparatus. It will count droplets, measure their diameters and velocities, and count freezing events. The script is fixed to analyse videos captured at a frame rate of 177fps of a channel width of 300 microns.

Required input files:

- ***.mp4 : The MP4 file is of the droplets passing through the LOC-NIPI. The script works best with MP4 files, AVI files can sometimes cause errors depending on the version of Python being used. VLC media player can be used to convert AVI to MP4.
- ***.png : The image is a screen capture of a frozen droplet. No particular file type is needed, PNG and JPEG work fine.

Output:

- data.csv : File containing all the gathered information about the droplets in the LOC-NIPI. Will automatically create and save file. File can be found in C:\\Users\\username.

Usage and requirements:

- 0) Install all needed packages, e.g., OpenCV.
- 1) Once ran, the GUI will appear, select 'Browse Image File' and select your image of the frozen droplet.
- 2) Then select 'Browse Video File' and select the LOC-NIPI recording.
- 3) A window for channel wall sampling will then appear, select the upper channel and lower channel wall three times each.
- 4) A window for background intensity calibration will appear, select a few points (at least one) in the region where the cold plate is expected to be.
- 5) A window for frozen droplet intensity calibration will appear, select a few points (at least one) on the droplet.
- 6) The first region of interest (ROI) selection window will appear, select the region where the cold plate lies. Then the second ROI selection window will appear, select the leftmost region of the channel, ideally around a third of the total channel length.
- 7) The program will begin running and write the found values to the data.csv file. Once the video has run its course, all values will be averaged and inserted in the last row of the data.csv file.

Questions: py19ua@leeds.ac.uk

"""

```
import numpy as np
import cv2
import tkinter as tk
from tkinter import filedialog
import time
import csv

class VideoPlayer(tk.Frame):
    def __init__(self, master):
        """Initialises all objects needed throughout the script.
        """
        super().__init__(master)
        self.master = master
        self.master.title("LOC-NIPI Analyser")
        self.video_source = ""

        #Block below creates a csv file called data and formats, if one exists with
        # name 'data.csv' then clear file and reformat.
        data = ['Frame count:', 'Time elapsed (seconds):', 'Total droplet count:',
                'Total frozen droplet count:', 'Live Velocity (microns/second)',
                'Live Diameter (microns):' ]
        with open('data.csv', mode='w', newline='') as file:
            file.truncate()
            writer = csv.writer(file)
            writer.writerow(data)

        self.speedavg=[]
        self.diameters=[]
        self.diameter=0

        # First window for droplet counter and velocity tracker.
        self.canvas = tk.Canvas(self.master, width=500, height=200)
        self.canvas.create_text(250, 10, text="Droplet Counter & Velocity Tracker")
        self.canvas.pack()

        # Second window for freeze detector.
        self.canvas1 = tk.Canvas(self.master, width=500, height=200)
        self.canvas1.create_text(250, 10, text="Frozen Droplet Counter")
        self.canvas1.pack()
```

```
# Creates first text box for droplet information.
self.text = tk.Text(self.master, width=50, height=10)
self.text.pack()

# Creates second text box for video information.
self.text1 = tk.Text(self.master, width=50, height=10)
self.text1.pack()

# Creates the background subtractor needed.
self.back_sub = cv2.createBackgroundSubtractorMOG2()

# Particle counter
self.particle_count = 0

# Temporary variable to hold distance travelled of each particle.
self.distance = 0

# Particle distance storage.
self.particle_distances = []

# Counts the frames each time it loops through the video.
self.framecount=[]

#####
# Objects needed for speed calculation.

# Counts the number of frames when there is a droplet present.
self.activeframes=0

# Stores the speed for each droplet.
self.speed=0

# Stores the distances in a list.
self.distancelist=[]
#####
# Objects needed for the frozen detector.

self.colors=[]
self.t=[]
```

```

self.countarray=[]
self.areaarray=[]

self.freezeSanityChecker=[]
self.vidtime=0

#####

def click_event(self,event, x, y, flags, params):
    '''
    Extracts information on click events, in this case: x and y coords.
    '''

    # Looks for left button clicks.
    if event == cv2.EVENT_LBUTTONDOWN:

        print(x, ' ', y)

        params.append(y)

        # Prints the coords on the cropped frame
        font = cv2.FONT_HERSHEY_SIMPLEX
        cv2.putText(self.cap_chen_width, str(x) + ', ' +
                    str(y), (x,y), font,
                    1, (255, 0, 0), 2)

        cv2.imshow('Channel Width Finder', self.cap_chen_width)

    return params

def channelPlotter(self):
    '''
    Processes the information extracted by the click_event function.
    '''
    by=[]
    _, self.cap_chen_width = self.cap.read()
    cv2.putText(self.cap_chen_width,
                str('Please select upper channel and lower channel 3 times'),
                (5, 40),
                cv2.FONT_HERSHEY_PLAIN, 1.7, (255, 0, 0),2)

```

```
cv2.putText(self.cap_chan_width, str('Press q once done'),
            (5, 70),
            cv2.FONT_HERSHEY_PLAIN, 1.7, (255, 0, 0),2)
cv2.imshow('Channel Width Finder', self.cap_chan_width)

cv2.setMouseCallback('Channel Width Finder', self.click_event, by)

# wait for a key to be pressed to exit
while True:
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# close the window
cv2.destroyAllWindows()

# The logic below sorts the list of pixel values and calculated the average width.
by.sort()
chan=[]
for i in range(6):

    if i > 2:
        break

    diff=by[-(i+1)]-by[i]
    chan.append(diff)

avgchan=sum(chan)/len(chan)
return avgchan

def browse_file(self):
    '''
    Browses for video file.
    '''
    self.video_source = filedialog.askopenfilename()
    self.play()

def browse_imagefile(self):
    '''
    Browses for image file.
    '''
```

```
self.image_source = filedialog.askopenfilename()

def imageProcessing(self):
    '''
    Image cleaning for droplet detection (diameter measurement etc).
    '''
    gray = cv2.cvtColor(self.framer, cv2.COLOR_BGR2GRAY)

    # applies the background subtraction
    foreground_mask = self.back_sub.apply(gray)

    # removes noise in the foreground mask, and blurs
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
    foreground_mask = cv2.morphologyEx(foreground_mask, cv2.MORPH_CLOSE, kernel)
    foreground_mask = cv2.medianBlur(foreground_mask, 9)

    # finds circles in the foreground mask.
    self.circles = cv2.HoughCircles(foreground_mask, cv2.HOUGH_GRADIENT,
                                     dp=1, minDist=100, param1=100, param2=10,
                                     minRadius=0, maxRadius=100)

    return foreground_mask

def estimate_droplet_diameter(self):
    '''
    Finds the largest circle from the Hough circle transform.
    '''
    # Finds the largest circle assuming that it's the droplet.
    max_circle = max(self.circles[0], key=lambda x: x[2])
    print(self.circles)
    print(max_circle)
    # Gets the diameter of the circle.
    self.diameter = 2 * max_circle[2]

def update_textbox(self):
    '''
    Updates the first text box with the latest droplet information.
    '''
    self.particle_count=int(self.particle_count)
    self.distance=int(self.distance)
```

```

self.speed=int(self.speed)

n1 = 'Droplet count: {}'.format(self.particle_count)
n2 = 'Distance travelled (px): {}'.format(self.distance)
n3 = 'Live Velocity (microns/second): {}'.format(self.speed)

self.text.delete('1.0', tk.END) # delete the current text
self.text.insert(tk.END, n1) # insert the new text
self.text.insert(tk.END, n2)
self.text.insert(tk.END, n3)

# logic to avoid errors div by 0 errors.
if len(self.diameters)>0:
    buffer=int(sum(self.diameters)/len(self.diameters))
    n4 = 'Live Average Diameter (microns): {}'.format(buffer)

if len(self.countarray)>0:
    n5 = 'Frozen Droplet Count: {}'.format(len(self.countarray))
    self.text.insert(tk.END, n5)

def update_textbox1(self):
    '''
    Updates the second text box with the latest video information.
    Updates csv with rows of new information.
    '''

    n1 = (
        'Frame count: '
        + str(len(self.framecount))
        + '/'
        + str(self.tot_frames)
        + '\n'
    )
    n2 = (
        'Time elapsed (seconds): '
        + str(int(len(self.framecount) / self.fps))
        + '/'
        + str(int(self.duration))
    )

```

```

self.text1.delete('1.0', tk.END) # delete the current text
self.text1.insert(tk.END, n1) # insert the new text
self.text1.insert(tk.END, n2) # insert the new text

# Conditional statements below to update and add rows to the
# csv file with new info.
# Logic to avoid div by zero errors.
if len(self.diameters)>0:

    data = [str(int(len(self.framecount)))+ '/'
            + str(int(self.tot_frames)),
            str(len(self.framecount)/self.fps) + '/' + str(self.duration),
            self.particle_count, len(self.countarray),
            int(self.speed), int(self.microndiameter)]

    #int(sum(self.diameters)/len(self.diameters))
else:
    data = [str(int(len(self.framecount)))+ '/'
            + str(int(self.tot_frames)),
            str(len(self.framecount)/self.fps) + '/' + str(self.duration),
            self.particle_count, len(self.countarray),
            int(self.speed), 0]

with open('data.csv', mode='a', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(data)

def on_mouse_click_background (self,event, x, y, flags, frame):
    '''
    Extracts information on click events for sampling the
    background colour.
    '''

    if event == cv2.EVENT_LBUTTONUP:
        sample = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        self.colors.append(sample[y,x].tolist())
        cv2.circle(frame,(x,y),12,(255,0,0),3)

def on_mouse_click_droplet (self,event, x, y, flags, frame):

```

```
'''
Extracts information on click events for sampling the
frozen droplet colour.
'''

if event == cv2.EVENT_LBUTTONUP:
    sample = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    self.colors.append(sample[y,x].tolist())

def eyeDropper(self,ans):
    '''
    Extracts information on click events for sampling the
    frozen droplet colour.
    '''
    self.colors=[]
    if ans == 'vid':
        _, frame = self.cap.read()

        while True:
            sample = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            if self.colors:
                cv2.putText(sample, str(self.colors), (450, 40),
                           cv2.FONT_HERSHEY_PLAIN, 2, (255, 0, 0),2)
            cv2.imshow('Background Colour Sampler (Eye Dropper Tool A)', sample)
            cv2.setMouseCallback('Background Colour Sampler (Eye Dropper Tool A)',
                                self.on_mouse_click_background, frame)

            # press q once done.
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

    if ans == 'pic':
        self.capImage = cv2.imread(self.image_source)
        frame = self.capImage

        while True:
            sample = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            if self.colors:
                cv2.putText(sample, str(self.colors), (450, 40),
```

```

        cv2.FONT_HERSHEY_PLAIN, 2,
        (255, 0, 0),2)
    cv2.imshow('Frozen Droplet Colour Sampler (Eye Dropper Tool B)', sample)
    cv2.setMouseCallback('Frozen Droplet Colour Sampler (Eye Dropper Tool B)',
        self.on_mouse_click_droplet, frame)

    # press q once done.
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    print(self.colors)
    ColorList=self.colors
    cv2.destroyAllWindows()

    return ColorList

def freezeImageProcessing(self):
    '''
    Image processing for frozen droplet detection.
    '''
    # This will blur the frames slightly to get rid of granularity and hence
    # an easier freeze detection.
    blur = cv2.medianBlur(self.framerffreeze, 9)

    # Calculations below find halfway point between frozen droplet colour and background.
    self.backgroundColor=sum(self.backgroundColorList)/len(self.backgroundColorList)
    self.frozenDropletColor=sum(self.frozenDropletColorList)/len(self.frozenDropletColorList)
    self.lim_back=int((self.frozenDropletColor+self.backgroundColor)/2)

    #This will change the frame to greyscale, and threshold based on the calibrated value.
    gray = cv2.cvtColor(blur, cv2.COLOR_BGR2GRAY)
    thresh = cv2.threshold(gray, self.lim_back ,255, cv2.THRESH_BINARY_INV)[1]
    return gray, thresh

def speedCalc(self):
    '''
    Speed calculation based on active frames and pixel distance. 1/176
    is the capture frame rate. 300 for the channel width being fixed
    at 300 microns.
    '''

```

```
self.micronsTravelled=(self.distance/self.avgC)*300
speed=self.micronsTravelled/(self.activeframes*(1/176))
```

```
return speed
```

```
def play(self):
    labelList=[]
    # Sets initial droplet distance to none.
    prevPt = None
    # Collects the capture data from the video pathway.
    self.cap = cv2.VideoCapture(self.video_source)
    # Gets number of total frames in video
    self.tot_frames = int(self.cap.get(cv2.CAP_PROP_FRAME_COUNT))
    # Gets total video time.
    self.fps = self.cap.get(cv2.CAP_PROP_FPS)
    self.duration = self.tot_frames/self.fps
    # Puts total frame count and total video duration in the second textbox.
    self.text1.insert(tk.END, f'Total frames:{self.tot_frames} Duration:{self.duration}')
```

#####

```
self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
self.avgC=self.channelPlotter()
print(self.avgC)
self.backgroundColorList=self.eyeDropper('vid')
self.frozenDropletColorList=self.eyeDropper('pic')
# Gets the region of interest coordiantes for the coldplate, and
# the furthest left channel region.
ret, firstFrame = self.cap.read()
self.rfreeze = cv2.selectROI('Cold Plate ROI (Frozen Droplet Detection Area)',firstFrame)
self.r = cv2.selectROI('Furthest Left ROI (Droplet Counting/Tracking)',firstFrame)

while True:
    # Reads self.cap.read() will read the frame from the video
    # that is being looped through.
    ret, frame = self.cap.read()
    if ret:
        # Process the frame here
        # self.framer is the far left ROI and self.framerfreeze is the cold plate area
        self.framer = frame[int(self.r[1]):int(self.r[1]+self.r[3]),
                               int(self.r[0]):int(self.r[0]+self.r[2])]
```

```

self.framerfreeze = frame[int(self.rfreeze[1]):int(self.rfreeze[1]
                                                    + self.rfreeze[3]),
                           int(self.rfreeze[0]):int(self.rfreeze[0]
                                                    + self.rfreeze[2])]

self.framecount.append(1)
self.update_textbox1()
foregroundMask=self.imageProcessing()
##### Circle Diameter Stuff #####
self.microndiameter=0
if self.circles is not None:
    # Conditional statement checks for detected circles, then processes.
    self.estimate_droplet_diameter()
    # Conversion to microns.
    self.microndiameter=(self.diameter/self.avgC)*300
    self.diameters.append(self.microndiameter)

    # Draws the largest circle and displays the diameter on the frame
    for circle in self.circles[0, :]:
        center = (int(circle[0]), int(circle[1]))
        radius = int(circle[2])
        cv2.circle(self.framer, center, radius, (0, 255, 0), 2)

#####

##### Freeze Event Counting Stuff #####
# Finds an extracts contours from the processed frame.
self.framerfreeze, thresh = self.freezeImageProcessing()
cnts = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cnts = cnts[0] if len(cnts) == 2 else cnts[1]

for c in cnts:
    #Uses the cv2 module to find the area of the contour.
    area = cv2.contourArea(c)
    # Square box of chan width is (self.avgC)^2 is max and the
    # avg size of droplet is 100micron diam. So do min of 50micron.
    if (self.avgC/6)**2 < area < (self.avgC)**2:
        #Draws the contouring line around the area.
        cv2.drawContours(self.framerfreeze,[c], 0, (36,255,12), 2)

```

```
#Once it has met criteria and fell into it statement, 1
# is added to the count array.
self.countarray.append(1)
print(self.countarray)

#Saved the area of the freezing event into the array.
self.areaarray.append(area)

# Double check it against the main count to make sure
# artefacts aren't detected.
self.freezeSanityChecker.append(self.particle_count)

# Conditional statement making sure that freezing event
# from only once frame was
# accepted, sometimes freezing events appear in more
# than one frame and to avoid
# double counting or accepting an artefact, it was
# compared to the main droplets count.
if (len(self.freezeSanityChecker) >= 2 and
    self.freezeSanityChecker[-1] == self.freezeSanityChecker[-2]):

    # Deletes the extra count and areas.
    del self.countarray[-1]
    del self.areaarray[-1]

#Puts the count of the freezing events on the frame.
self.framerfreeze = cv2.putText(self.framerfreeze,
                                str(len(self.countarray)), (520, 40),
                                cv2.FONT_HERSHEY_PLAIN, 2, (255, 0, 0),2)

self.update_textbox()
#####

##### Droplet Detection Counting Stuff #####

# Labels all the connected components in the foreground mask.
(num_labels, labels, stats, centroids
) = cv2.connectedComponentsWithStats(foregroundMask, connectivity=8)
```

```
#####UPTIL HERE#####14/4/23
```

```
labelList.append(num_labels)
```

```
if len(self.framecount) != 1 and num_labels >1:
```

```
    x, y, w, h, area = stats[1]
```

```
    # Draws a bounding rectangle around the droplet.
```

```
    cv2.rectangle(self.framer, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

```
    # Counts the amount of frames where there is a present
```

```
    self.activeframes +=1
```

```
    # Get the center of the droplet
```

```
    centerX, centerY = int(centroids[1][0]), int(centroids[1][1])
```

```
    cv2.circle(self.framer, (centerX, centerY), 5, (0, 0, 255), -1)
```

```
    # Condition to check if a new particle has entered the frame.
```

```
    # else: it will assume the particle isn't a new particle entering the frame.
```

```
    if prevPt is None:
```

```
        # increment particle count
```

```
        self.particle_count += 1
```

```
        self.update_textbox()
```

```
        # Initialises distance at 0 and adds to a list.
```

```
        self.distance = 0
```

```
        self.distancelist.append(self.distance)
```

```
    else:
```

```
        # Increments the distance value by calculating the distance
```

```
        # the droplet moved from the previous previous point (prevPt).
```

```
        self.distance += np.sqrt(
```

```
            (centerX - prevPt[0])**2 + (centerY - prevPt[1])**2
```

```
        )
```

```
        cv2.putText(self.framer, "Distance: {:.2f} pixels".format(self.distance),
```

```
            (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
```

```
            0.6, (0, 0, 255), 2)
```

```
##### Speed Calculation Stuff #####
```

```
self.speed=self.speedCalc()
```

```
# Calculates the speed between frames then adds to list of speed values.
```

```
self.speedavg.append(self.speed)
```

```

        # Updates the live output and saves the distance moved in a list.
        self.update_textbox()
        self.distancelist.append(self.distance)
    # Assigns a previous point after droplet parameter processing.
    prevPt = (centerX, centerY)

# Checks if the droplet has left the frame.
if len(labelList)>2 and labelList[-1]==1 and labelList[-2]>1 :
    # store the final total distance traveled by the droplet
    self.particle_distances.append(self.distance)
    print(
        f'Particle {self.particle_count} left the frame, '
        f'distance traveled: {self.distance} pixels'
    )
    # Resets all the information from the last droplet.
    prevPt = None
    self.activeframes=0
    self.speed=0

cv2.waitKey(1)
# Allows the video feeds to be displayed in the GUI.
photo = tk.PhotoImage(data=cv2.imencode('.png', self.framer)[1].tobytes())
self.canvas.create_image(250,50, image=photo, anchor=tk.CENTER)
photo1 = tk.PhotoImage(data=cv2.imencode('.png', foregroundMask)[1].tobytes())
self.canvas.create_image(250,120, image=photo1, anchor=tk.CENTER)

# Same as above but for the second canvas.
photo0 = tk.PhotoImage(data=cv2.imencode('.png', self.framerfreeze)[1].tobytes())
self.canvas1.create_image(250,50, image=photo0, anchor=tk.CENTER)
photo01 = tk.PhotoImage(data=cv2.imencode('.png', thresh)[1].tobytes())
self.canvas1.create_image(250,120, image=photo01, anchor=tk.CENTER)
self.master.update()

# Once all frames in the video have been looped through, the final
# totals/averages are added to the data.csv file.
else:
    data = [str(int(len(self.framecount)))+ '/' + str(int(self.tot_frames)),
            str(len(self.framecount)/self.fps) + '/' + str(self.duration),
            self.particle_count,len(self.countarray),

```

```
        int(sum(self.speedavg)/len(self.speedavg)),  
        int(sum(self.diameters)/len(self.diameters))]  
    with open('data.csv', mode='a', newline='') as file:  
        writer = csv.writer(file)  
        writer.writerow(data)  
    break
```

```
root = tk.Tk()  
root.geometry("400x800")  
player = VideoPlayer(root)  
player.pack()
```

```
browse_button = tk.Button(root, text="Browse Video File", command=player.browse_file)  
browse_button.pack()
```

```
browse_button1 = tk.Button(root, text="Browse Image File", command=player.browse_imagefile)  
browse_button1.pack()
```

```
root.mainloop()
```

REFERENCES

- [1] M. D. Tarn, S. N. Sikora, G. C. Porter, B. V. Wyld, M. Alayof, N. Reicher, A. D. Harrison, Y. Rudich, J. U. Shim, and B. J. Murray, “On-chip analysis of atmospheric ice-nucleating particles in continuous flow,” *Lab on a Chip*, vol. 20, pp. 2889–2910, 8 2020.
- [2] S. Huang, W. Hu, J. Chen, Z. Wu, D. Zhang, and P. Fu, “Overview of biological ice nucleating particles in the atmosphere,” *Environment International*, vol. 146, p. 106197, 1 2021.
- [3] G. Vali, P. J. DeMott, O. Mohler, and T. F. Whale, “Technical note: A proposal for ice nucleation terminology,” *Atmospheric Chemistry and Physics*, vol. 15, pp. 10263–10270, 9 2015.
- [4] Z. A. Kanji, L. A. Ladino, H. Wex, Y. Boose, M. Burkert-Kohn, D. J. Cziczo, and M. Kramer, “Overview of ice nucleating particles,” *Meteorological Monographs*, vol. 58, pp. 1.1–1.33, 1 2017.
- [5] A. S. Basu, “Droplet morphometry and velocimetry (dmv): a video processing software for time-resolved, label-free tracking of droplet parameters,” *Lab on a Chip*, vol. 13, pp. 1892–1901, 4 2013.
- [6] B. Ribeiro, D. Martins, A. onio, A. Neves, A. Pinho, and D. Ieeta, “Arbitrary ball detection using the circular hough transform,” 04 2023.
- [7] M. Szelag, “Evaluation of cracking patterns in cement composites-from basics to advances: A review,” *Materials*, vol. 13, 6 2020.
- [8] M. Zantow, R. Dendere, and T. S. Douglas, “Image-based analysis of droplets in microfluidics,” *Annual International Conference of the IEEE Engineering in*

REFERENCES

- Medicine and Biology Society. IEEE Engineering in Medicine and Biology Society. Annual International Conference*, vol. 2013, pp. 1776–1779, 2013.
- [9] G. P. Rutkowski, I. Azizov, E. Unmann, M. Dudek, and B. A. Grimes, “Microfluidic droplet detection via region-based and single-pass convolutional neural networks with comparison to conventional image analysis methodologies,” *Machine Learning with Applications*, vol. 7, p. 100222, 2022.
- [10] P. Hadikhani, N. Borhani, S. M. H. Hashemi, and D. Psaltis, “Learning from droplet flows in microfluidic channels using deep neural networks,” *Scientific Reports*, vol. 9, no. 1, p. 8114, 2019.
- [11] D. J. Cziczo, K. D. Froyd, C. Hoose, E. J. Jensen, M. Diao, M. A. Zondlo, J. B. Smith, C. H. Twohy, and D. M. Murphy, “Clarifying the dominant sources and mechanisms of cirrus cloud formation,” *Science*, vol. 340, pp. 1320–1324, 6 2013.
- [12] M. Wondie, “Modeling cloud seeding technology for rain enhancement over the arid and semiarid areas of ethiopia,” *Heliyon*, vol. 9, p. e14974, 4 2023.
- [13] A. Laaksonen and J. Malila, “Ice nucleation,” *Nucleation of Water*, pp. 209–248, 2022.
- [14] A. Lagriffoul, J. L. Boudenne, R. Absi, J. J. Ballet, J. M. Berjeaud, S. Chevalier, E. E. Creppy, E. Gilli, J. P. Gadonna, P. Gadonna-Widehem, C. E. Morris, and S. Zini, “Bacterial-based additives for the production of artificial snow: What are the risks to human health?,” *Science of The Total Environment*, vol. 408, pp. 1659–1666, 3 2010.
- [15] L. Ding and A. Goshtasby, “On the canny edge detector,” *Pattern Recognition*, vol. 34, pp. 721–725, 3 2001.
- [16] N. Otsu, “Threshold selection method from gray-level histograms.,” *IEEE Trans Syst Man Cybern*, vol. SMC-9, pp. 62–66, 1979.
- [17] J. Y. Tinevez, N. Perry, J. Schindelin, G. M. Hoopes, G. D. Reynolds, E. Laplantine, S. Y. Bednarek, S. L. Shorte, and K. W. Eliceiri, “Trackmate: An open and extensible platform for single-particle tracking,” *Methods*, vol. 115, pp. 80–90, 2 2017.

REFERENCES

- [18] J. M. Martinez-Otzeta, I. Rodriguez-Moreno, I. Mendiadua, and B. Sierra, “Ransac for robotic applications: A survey,” *Sensors 2023, Vol. 23, Page 327*, vol. 23, p. 327, 12 2022.